# JAYPEE INSTITUTE OF INFORMATION TECHNOLOGY, NOIDA




## AI&ML PROJECT SYNOPSIS

## ECHOPATH

## B.TECH CSE 3RD SEMESTER

**SUBMITTED TO:**
*DR. SAYANI GHOSH*

**ASSIGNED TO:**
*HARSH JHA: 2401030141*
*JAYDEEP SINGH: 2401030133*
*ARNAV TYAGI: 2401030139*

DATE: 27/ 01/ 2026

# 1. Introduction

Music streaming apps can generate playlists, but many playlists still feel random because they do not explicitly optimize for smooth transitions between tracks while also respecting constraints like duration, mood progression, genre boundaries, and personal preference. This project proposes EchoPath, an AI-driven playlist generator that treats songs as nodes in a graph and uses search algorithms plus machine learning to build playlists that sound coherent, gradual, and intentional.

**The system combines three ideas into one pipeline:**

1. Search-based playlist construction (BFS/DFS/UCS/IDS, plus A* and IDA*) using song-to-song similarity costs.
2. Unsupervised learning to discover mood clusters and structure transitions.
3. Supervised and deep learning to predict genre or mood from audio (CNN on mel-spectrograms) and to model user preference signals (skip/like prediction).

The final output is a playlist that starts from a chosen song and reaches a target mood or genre, while keeping transitions smooth and constraints satisfied.

# 2. Problem Statement

Given a library of songs with audio features, generate a playlist of length $N$ (or total duration $T$) starting from a user-selected track such that:

- Consecutive songs have high transition smoothness (low similarity cost).
- The playlist satisfies constraints such as target mood, target genre, explicit content filter, artist repetition limits, and tempo progression rules.
- The playlist is personalized using predicted "like/skip" probabilities when user preference data is available.

# 3. Objectives

1. Build a song feature pipeline using Python for extracting and organizing audio features and metadata.
2. Construct a similarity graph of songs and implement multiple search strategies: BFS, DFS, UCS, IDS and informed methods: A*, IDA*, plus a hill-climbing refinement stage.
3. Apply k-means clustering on audio features to discover mood groupings and use clusters to guide playlist structure.
4. Train supervised models (Logistic Regression, SVM, Random Forest, MLP) to predict preference signals such as skip/like or to estimate "fit" to a mood.
5. Train and compare CNN architectures (LeNet-style baseline, deeper VGG-style, ResNet-style) on mel-spectrograms for genre or mood classification.
6. Perform a comparative analysis of algorithms and models using measurable metrics and a consistent benchmarking setup.

# 4. Proposed Methodology

## 4.1 Data Collection and Preprocessing

- Use an open dataset such as FMA (Free Music Archive) for audio files and metadata, or GTZAN for genre classification experiments.

- Standardize audio (sampling rate, mono conversion), clean metadata, and build a consistent dataset split for ML/DL evaluation.
- Extract features using audio processing libraries (e.g., MFCCs, chroma, spectral contrast, tempo, energy statistics). Store results in structured tables for analysis.

### 4.2 Song Similarity Graph Construction

- Represent each song as a feature vector.
- Define a distance metric (cosine distance or Euclidean distance after normalization).
- Build a k-nearest-neighbor graph where edges represent "transition cost" between songs.

### 4.3 Playlist Generation via Search

- Model the playlist as a pathfinding problem: start node is the seed song, goal is reaching a target mood cluster/genre (or meeting constraints at length N).
- Implement and compare:
    - Uninformed search: BFS, DFS, UCS, IDS
    - Informed search: A* and IDA* using heuristics such as distance to target cluster centroid or predicted mood probability gap
    - Hill climbing: post-process the generated playlist to locally swap or replace tracks to reduce total cost while maintaining constraints

### 4.4 Unsupervised Mood Discovery

- Apply k-means on audio features (and optional embeddings from a trained model) to create mood clusters.
- Interpret clusters using feature profiles (tempo, energy, brightness) and use them as intermediate "checkpoints" for smooth mood transitions.

### 4.5 Supervised Preference Modeling

- If a labeled signal exists (like/skip) use classical ML models to predict preference from audio features and metadata.
- If explicit preference labels are not available, simulate labels with a rule-based proxy (example: treat high similarity to a user's favorites as a positive label) and clearly report it as a proxy in the evaluation.

### 4.6 Deep Learning with CNN on Spectrograms

- Convert audio to mel-spectrogram images.
- Train multiple CNN variants in TensorFlow/Keras:
    - LeNet-style baseline
    - VGG-style deeper CNN
    - ResNet-style (with residual blocks)
- Compare performance and training behavior, then use predicted mood/genre probabilities as guidance signals for A* heuristics or constraint satisfaction.

## 5. Evaluation Plan

- Search performance: runtime, nodes expanded, memory usage estimates, path cost (smoothness), constraint satisfaction rate.
- Playlist quality metrics: average transition cost, variance of tempo/energy, diversity score (artist and genre diversity), and target achievement rate (ending mood/genre).

- ML metrics: accuracy, precision/recall/F1 for classification; ROC-AUC for binary preference prediction.
- CNN metrics: accuracy/F1 on genre or mood classification; model comparison across architectures; confusion matrix and error analysis.
- Final comparative analysis will connect algorithm choices to measurable outcomes.

# 6. Tools and Technologies

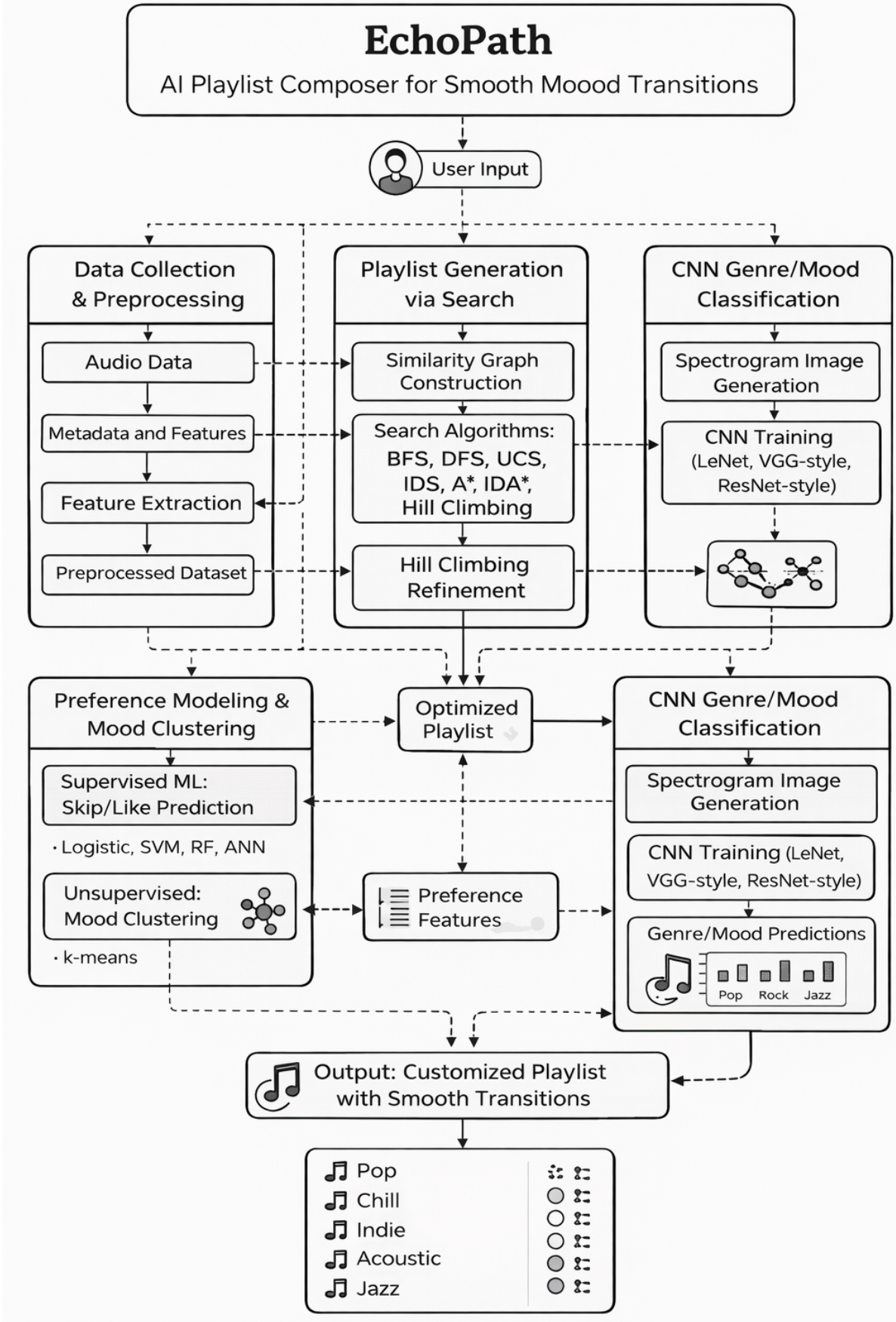
Python, NumPy, Pandas, SciPy, scikit-learn, TensorFlow/Keras, librosa (audio processing), matplotlib for plots, and a lightweight UI (web app) for dem o.

# 7.Expected Deliverables

1. Working playlist generator with selectable algorithms (BFS/DFS/UCS/IDS/A*/IDA* + hill climbing).
2. Mood clustering module with visualizations and cluster summaries.
3. Classical ML preference predictor and evaluation report.
4. CNN-based genre/mood classifier with architecture comparison.
5. Final report with experiments, benchmarks, analysis and a demo showing constraint-based playlist creation.

# 8.References


1. Scikit-learn: https://scikit-learn.org/stable/
2. ANN: https://simple.wikipedia.org/wiki/Artificial_neural_network
3. CNN: https://en.wikipedia.org/wiki/Convolutional_neural_network
4. librosa: https://librosa.org/doc/latest/index.html
5. scipy: https://docs.scipy.org/doc/scipy/
6. pandas: https://pandas.pydata.org/docs/


EchoPath
AI Playlist Composer for Smooth Moood Transitions
User Input
Data Collection & Preprocessing
Audio Data
Metadata and Features
Feature Extraction
Preprocessed Dataset
Playlist Generation via Search
Similarity Graph Construction
Search Algorithms: BFS, DFS, UCS, IDS, A*, IDA*, Hill Climbing
Hill Climbing Refinement
CNN Genre/Mood Classification
Spectrogram Image Generation
CNN Training (LeNet, VGG-style, ResNet-style)
Preference Modeling & Mood Clustering
Supervised ML: Skip/Like Prediction
• Logistic, SVM, RF, ANN
Unsupervised: Mood Clustering
• k-means
Optimized Playlist
Preference Features
CNN Genre/Mood Classification
Spectrogram Image Generation
CNN Training (LeNet, VGG-style, ResNet-style)
Genre/Mood Predictions
Pop
Rock
Jazz
Output: Customized Playlist with Smooth Transitions
Pop
Chill
Indie
Acoustic
Jazz